

Instituto Politécnico Nacional

Centro de Investigación en Computación (CIC)

Departamento de diplomados y extensión profesional

Diplomado en Python y Ciencia de Datos

Práctica 2

Introducción a las redes neuronales multicapa

Profesor Alan Badillo Salas

Semana 1. Agosto 2026.

Introducción

En la ciencia de datos, el aprendizaje profundo ha tomado relevancia gracias a las redes neuronales multicapa, las cuales son capaces de generar modelos no lineales complejos que son capaces de predecir respuestas, generando así un mecanismo de aprendizaje supervisado útil y de buen rendimiento, una red neuronal multicapa consta principalmente de tres tipos capas:

1. **Capa de entrada** - Esta no contiene neuronas, sino que codifica el vector de entrada $x^{(i)}$ para cada observación.
2. **Capas ocultas** - Estas definen un conjunto de neuronas con una misma función de activación, la cual se alimenta del vector de la capa anterior, por ejemplo, $x^{(i)}, h^{(1)}, h^{(2)}, \dots$, produciendo un vector de respuesta intermedia $h^{(l)}$.
3. **Capa de salida** - Esta capa considera una neurona para cada respuesta requerida, y donde la activación de esa respuesta debe permitir construir el mismo espacio que la respuesta, por ejemplo, en respuestas abiertas se puede tener una activación lineal y en respuestas cerradas una activación logística. Cada neurona en la capa de salida se considera el predictor de respuesta $\hat{y}_j$ para la j -ésima salida.

Al igual que un único perceptrón, la red neuronal requiere de una función de pérdida asociada y un optimizador, esto formará capas densas que serán optimizadas de forma similar al perceptrón, pero retropropagando el gradiente (jacobiano) del error para cada capa oculta $h^{(l)}$. A este método se le conoce como la **retropropagación (backpropagation)**.

El modelo matemático para definir la siguiente capa oculta de activación uniforme y matriz de pesos $B^{(l+1)}$ es:

$$h^{(l+1)} = \Phi_{l+1}(h^{(l)}, B^{(l+1)})$$

donde $\Phi_{l+1}(h^{(l)}, B^{(l+1)})$ es el vector de la respuesta intermedia $h^{(l+1)}$ que será el vector de entrada para la siguiente capa. Así podemos pensar $h^{(0)} = x^{(i)}$ como la capa oculta almohadilla, $h^{(1)} = \Phi_1(h^{(0)}, B^{(1)})$ la primera capa oculta con una matriz de pesos $B^{(1)} = (B_1^{(1)}, B_2^{(1)}, \dots, B_{l_1}^{(1)})$, donde cada $B_\nu^{(1)}$ es el vector de pesos asociado a la neurona ν , por eso, cada capa oculta tiene una matriz de pesos, equivalente a una colección de vectores de pesos, uno para cada neurona (donde la neurona se entiende como un perceptrón individual). Luego se continúa así con $h^{(2)} = \Phi_2(h^{(1)}, B^{(1)})$, haciendo que la segunda capa oculta tome como entrada la primera capa oculta y su función de activación Φ_2 . Este proceso permite encadenar todas las capas que se alimentan desde $h^{(0)} = x^{(i)}$ y llegan a $h^{(s)}$ que es esencialmente la última capa de salida.

Para lograr la optimización se requieren dos piezas fundamentales de derivación, la primera es el gradiente de $\Phi_{l+1}(h^{(l)}, B^{(l+1)})$ respecto a las $h^{(l)}$, el cual es

un vector de gradientes conocido como el Jacobiano $J_{h^{(l)}}$. Luego el gradiente de $\Phi_{l+1}(h^{(l)}, B^{(l+1)})$ respecto a los pesos $B^{(l+1)}$ que también es un Jacobiano $J_{B^{(l+1)}}$. Teniendo, entonces:

$$J_{h^{(l)}} = \frac{\partial h^{(l+1)}}{\partial h^{(l)}} = \frac{\partial \Phi_{l+1}}{\partial h^{(l)}}$$

Y también

$$J_{B^{(l+1)}} = \frac{\partial h^{(l+1)}}{\partial B^{(l+1)}} = \frac{\partial \Phi_{l+1}}{\partial B^{(l+1)}}$$

Y así, el punto crítico será cómo retropropagar el error, el cuál se divide en dos piezas, la primera es $\varepsilon^{(s)}$ que es el error en la capa de salida, medido a través de la función de pérdida $L(\varepsilon^{(s)})$ y $\varepsilon^{(l)}$ que es el error que se retropropaga a la capa anterior, teniendo así:

$$\varepsilon^{(s)} = -\frac{\partial L}{\partial h^{(s)}}$$

Y también,

$$\varepsilon^{(l)} = J_{h^{(l)}}^\top \varepsilon^{(l+1)}$$

donde la última capa oculta conecta $h^{(l+1)=h^{(s)}}$ que es la capa de salida.

Para el caso del error cuadrático medio $L = \frac{1}{2} \|\varepsilon\|^2 = \frac{1}{2} \|y - h^{(s)}\|^2$, tendríamos:

$$\varepsilon^{(s)} = -\frac{\partial L}{\partial h^{(s)}} = -\varepsilon \frac{\partial \varepsilon}{\partial h^{(s)}} = \varepsilon(-1) = \varepsilon = y - h^{(s)}$$

Finalmente, para que la retropropagación esté completa tenemos la actualización de pesos por descenso del gradiente como:

$$B^{(l+1)*} = B^{(l+1)} + \eta J_{B^{(l+1)}}^\top \varepsilon^{(l+1)}$$

Completando así el modelo de optimización de la red neuronal multicapa.

En esta **Práctica 2**, no implementaremos dicho modelo de optimización, sino que usaremos la librería de *Keras* de *Tensorflow* para optimizar una red neuronal multicapa mediante el modelo secuencial de capas densas.

Resuelve los dos problemas teóricos y responde las preguntas de la práctica. En un documento **PDF** con una portada que contenga la identidad del *Instituto Politécnico Nacional*, el *Centro de Investigación en Computación* y el *Departamento de diplomados y extensión profesional*, con tu nombre completo, correo, así como lugar y fecha, documenta los resultados de la práctica y envíalos por correo a **badillosalas@outlook.com** con el título **Diplomado PyCD 2026 - Práctica 2**. No es necesario poner cuerpo en el correo o adjuntar más archivos, solo el documento PDF es suficiente. El documento PDF se debe llamar *pycd26-SIGLAS.pdf* con las **SIGLAS** formadas por tu nombre y apellidos.

1. Problema 1 - Parámetros de entrenamiento

Una red neuronal multicapa define matrices de pesos en cada capa y por cada neurona, así si tenemos una red neuronal de dos capas ocultas, donde la primera capa tiene 3 neuronas y la segunda 2 neuronas, y una capa de salida de 1 neurona tendríamos la arquitectura $3 \times 2 \times 1$, si la entrada $x^{(i)}$ es un vector de entrada con 10 características, entonces el cálculo del total de parámetros quedaría como:

1. **Capa oculta 1** - $(10 + 1) \cdot 3$ considerando 10 entradas más el bias
2. **Capa oculta 2** - $(3 + 1) \cdot 2$ considerando 3 entradas (salidas de la primera capa) más el bias
3. **Capa de salida** - $(2 + 1) \cdot 1$ considerando 2 entradas (salidas de la segunda capa) más el bias
4. **Total de parámetros** - $33 + 8 + 3 = 44$

Entonces necesitaríamos un total de 44 β para entrenar una red neuronal multicapa de 10 características y 2 capas ocultas de 3×2 neuronas y una capa de salida de 1 neurona.

Completa la siguiente tabla para calcular el total de parámetros de aprendizaje en diferentes arquitecturas:

Entradas	Capas ocultas	Salidas	Total de Parámetros
10	3×2	1	44
8	3×2	1	-
10	3×3	1	-
8	3×3	1	-
10	$5 \times 4 \times 2$	1	-
8	$5 \times 4 \times 2$	1	-
30	$128 \times 64 \times 32$	1	-
30	$128 \times 64 \times 32$	10	-
784	32×32	10	-

1.1. Preguntas de análisis del Problema 1

Responde las siguientes preguntas:

1. ¿Qué representa cada parámetro de entrenamiento?
2. ¿Entre más capas habrá un mejor entrenamiento? ¿Por qué?
3. ¿Si hay pocos datos es mejor más capas o menos capas?
4. ¿Cuál es la mejor arquitectura para 784 características de entrada?

2. Problema 2 - Operador convolucional simple

En el siguiente problema hay dos matrices, una grande de datos que representa las características observadas en una imagen de 28×28 y otra matriz más pequeña que representa un filtro binario de 5×5 . Realiza el siguiente procedimiento para construir una matriz convolucionada mediante un método simple que simplifique la matriz grande y la reduzca en el promedio activado:

1. Ubica la matriz pequeña al inicio de la matriz grande
2. Multiplica esa parte de la matriz grande por la matriz pequeña
3. Obtén la suma de los valores de la matriz multiplicada
4. Pon el resultado como en una nueva matriz en esa posición
5. Avanza la matriz pequeña dos posiciones a la derecha
6. Repite la multiplicación y suma y colócala como la segunda posición en la matriz resultante
7. Repite sobre la misma fila
8. Al llegar a la última posición válida desplaza la matriz pequeña al principio 3 filas abajo
9. Multiplica la matriz pequeña por la parte de la matriz grande siguiendo el mismo procedimiento para generar la segunda fila de la matriz resultante
10. Termina de construir la matriz resultante

Al aplicar el algoritmo, el barrido de la matriz pequeña sobre la matriz grande en cada fila desplazando de dos en dos la posición generará una fila en la matriz resultante, luego la segunda fila con un barrido tres filas abajo y así hasta terminar. Esta matriz resultante representará la imagen transformada por el filtro y tendrá una dimensión menor.

Matriz de la imagen: Matriz del filtro:

[illegible]

$$\begin{pmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{pmatrix} \quad (1)$$

2.1. Preguntas de análisis del Problema 2

Responde las siguientes preguntas:

1. ¿Qué dimensiones tienen la matriz grande la imagen y la matriz pequeña del filtro?
2. ¿Qué dimensión tiene la matriz resultante?
3. Investiga o con ayuda de la IA. ¿Qué es un *stride*?
4. Investiga o con ayuda de la IA. ¿Cuál es el verdadero operador convolucional utilizado en la *CNN* y cómo opera?
5. Investiga o con ayuda de la IA. ¿Qué formas de reducir la convolución a un solo valor existen?

3. Preguntas de análisis de la práctica

Sobre el código de la práctica, responde las siguientes preguntas de análisis:

1. Para una imagen de 64×64 píxeles ¿Cuál es la dimensión del vector de aprendizaje si se aplana?
2. ¿Cuál es el primer paso para entrenar una red neuronal con imágenes?
3. ¿Por qué se normalizaron las imágenes a 64×64 píxeles?
4. ¿Por qué no se usan las imágenes a color y se convirtieron a escala de grises?
5. ¿Qué pasó con las imágenes que no eran cuadradas? ¿Cuál fue el procedimiento para cuadrarlas?
6. ¿Cómo se recuperó cada imagen como un vector de $64 * 64$?
7. ¿Por qué se separaron las imágenes en X_a , X_b , Y_a y Y_b ? ¿Cuál fue la estrategia?
8. ¿Qué porcentaje de exactitud alcanzó una red neuronal multicapa? ¿Consideras que fue suficiente?
9. ¿Cómo se aplicó el modelo secuencial de *Keras* a para construir la red neuronal convolucional?
10. ¿Qué porcentaje de exactitud alcanzó una red neuronal convolucional? ¿Consideras que fue suficiente?
11. ¿Cómo se obtuvo la predicción de imágenes de prueba?
12. ¿Cómo se visualizaron los resultados alcanzados por la predicción en las 14 imágenes de prueba?
13. Si le tomaras una foto a tu mascota o un perro o gato cualquiera desde tu celular, ¿Cómo podrías obtener la predicción?
14. ¿Cuántas respuestas devuelve la red neuronal como predicción? ¿En qué espacio?
15. Investiga o con ayuda de la IA. ¿Qué función de activación para la salida sería mejor *sigmoid* o *softmax*? ¿Por qué?
16. ¿Qué sesgo tiene la red neuronal al aprender solo imágenes de gatos y perros para predecir si es un gato o un perro?
17. ¿Cómo podríamos aumentar el número de imágenes de entrenamiento y validación?
18. Investiga o con ayuda de la IA. ¿Qué es un operador convolucional? ¿Cómo funciona?

19. ¿Por qué las convoluciones se tienen que aplanar para dar una respuesta?
20. ¿Qué aprende la red neuronal convolucional al final para generar la respuesta?

4. Método de evaluación

Cada pregunta respondida correctamente tendrá valdrá 1 punto, y cada respuesta aproximada solo medio punto. Hay 30 puntos totales en esta práctica. No hay fecha de entrega programada, tienen el resto del módulo para resolver la práctica, serán 3 prácticas totales y dos de ellas deberán aprobarlas con una calificación mayor a 6/10.

Mucho éxito en su práctica :D